

SOTA.md — Road to SOTA

실측으로 신기록이 나올 때만 갱신하는 문서. 실패한 실험과 중간 시도는 RESULTS.md에 있다. 여기는 "어떻게 여기까지 왔는가"의 서사만 남긴다. 추후 report의 원본이 된다.

이 문서에 오르는 숫자는 RNGD 실측뿐이다. makespan은 §4에 참고로만 둔다 — 2026-09-09의 첫 실측이 makespan 사다리 27칸 중 위쪽 절반을 통째로 무효화했다(§3.4).

1. 현재 SOTA

항목	값
브랜치	<code>V204_submit</code> (= V194 + attn_out 256 B 런(V198) + qkv·ffn 스테이징 store 병합(V199); 측정본은 <code>V200_production</code>)
커밋	<code>3c381d2</code>
측정	2026-09-10 05:12 UTC, 공식 채점(moa-submitter <code>46bc9eed</code>)
정확도	3/3 PASS (공식 + Arena 50/50)

커널	공식 baseline	V204 공식	speedup
<code>sliding_project_qkv</code>	250,514	101,132	2.478×
<code>sliding_attention_output</code>	404,633	56,845	7.118×
<code>decoder_feedforward</code>	3,703,473	317,445	11.666×
기하평균			5.9032

직전 공식 SOTA는 V165의 5.7576. **+2.5%**. Arena 실측(잡 14개)은 qkv 100.3k / attn_out 49.9k / ffn 310.1k = 6.229였는데 공식 draw는 attn_out이 56,845로 높게 나왔다(같은 코드의 Arena 범위 47.9k~52.0k) — cold 1회 draw의 분산이지 코드 차이가 아니다. 재제출로 좋은 draw를 노릴 수 있으나 제출 예산은 하루 5회다(RULES §5.4).

1.0a Arena SOTA 후보 (공식 미제출): `V200_production` (2026-09-10 04:40 UTC)

메모리 대역폭 캠페인(V196~V203, Arena 41슬롯)의 결과. V194 위에 실측으로 확인한 두 변경:

변경	커널	한 잡 안 A/B
O-weight 열 분할 8청크(512 B 런) → 16청크(256 B 런) , 슬라이스당 60행 → 120행	attn_out	5/5 잡 승, -3,707 (V200) / 4/4 잡 warm -3,450 · cold -5,470 (V198)
스테이징 store 2개 → 1 DMA 명령	ffn	6/6 잡 승, -11,078 (-3.5%)
같은 store 병합	qkv	중립(평균 +447, 노이즈) — 해가 없어 유지

실측 (V200, 잡 8개 추가, 커널당 n=24/16): qkv **100,314** · attn_out **49,928** · ffn **310,088** → 기하평균 **6.229** (cold-first median 기준 6.19). 같은 방식의 V194는 5.85~5.96 → **+5%**.

왜 통했는가. (1) V197이 DMA 비용 곡선을 실측했다: **256 B 런이 최적**(618 B/cycle)이고 그보다 짧으면 정확히 2× 벌점, 길어도 완만히 나빠진다. attn_out은 분할 축 Qs=4096이 2의 거듭제곱이라 256 B 정렬이 성립한다 (H=3840인 다른 두 커널은 불가). (2) DMA 명령 병합의 가치는 명령 수가 아니라 **그 명령이 커널 선두에서 weight 스트림을 막고 있는지**로 정해진다(V201이 반례).

공식 리더보드 제출은 사용자 지시가 있을 때만(RULES §5.4). 제출 전까지 공식 SOTA는 V165 5.7576 그대로.

1.0a-prev 직전 Arena 후보: `V194_production` (2026-09-10 00:20 UTC)

V182(= V165 + V181 ffn) 위에 실측으로 확인한 두 변경을 더한 것. 커밋 `V194_production`.

변경	커널	한 잡 안 A/B (V193, 변형당 n=3 × 3잡)
RoPE cos/sin 스테이징 버퍼 공유 + cos 곱을 sin 덧셈에 융합	qkv	5잡 평균 -3,883 (-3.6%) , 5/5 잡
O-weight 행 분할 48/12 → 44/16	attn_out	4/5 잡 승, 이상치 제외 평균 -330; V188 전용 스왑에서는 cold -2.8k로 두 잡 모두 최저
(V182에서 이미) up/gate를 슬라이스당 30행 × H 전체로	ffn	317,947 vs 348,967 → -31,020 (-8.9%)

잡간 절대값은 신뢰하지 말 것 (RULES §5.4): 같은 코드가 잡에 따라 attn 51.7k~60.3k, ffn 311.9k~324.2k로 흔들린다. 판정은 한 잡 안 warm median A/B로만 한다. V165의 공식 draw에 위 델타를 적용한 기대 공식 점수 ≈ **6.05** (qkv 101,661 / attn_out 52,574 / ffn 318,000).

공식 리더보드 제출은 사용자 지시가 있을 때만(RULES §5.4). 제출 전까지 공식 SOTA는 V165 5.7576 그대로.

1.0b 직전 Arena 후보: `V182_ffn_full_rows_production` (2026-09-09 18:35 UTC)

V165 + ffn up/gate를 슬라이스당 30행 × H 전체(연속 1세그먼트 로드)로 바꾼 V181의 ffn(`stage_x_hi_lo_hbm_full` + `feedforward_v181`). 커밋 `53ba401`. Arena job 15622/15623/15624(커널당 5회 × 3잡) **45/45 PASS**, cold median **qkv 110,234 / attn_out 52,928 / ffn 319,094** → 기하평균 **5.86** (V165를 같은 Arena-cold 기준으로 재면 5.72; 공식 draw 기대치 ≈ 5.9). ffn -8.6%, qkv·attn_out은 V165와 같은 분포. **공식 리더보드 제출은 사용자 지시가 있을 때만**(RULES §5.4). 제출 전까지 공식 SOTA는 V165 그대로.

1.1 공식 리더보드 (moa-submitter)

날짜	submission	코드	qkv	attn_out	ffn	score	순위
2026-09-09 15:01 UTC	23cc6523	V82 07a187f	155,419	52,492	349,760	5.0860	2위 (1위 #905: 5.667 = 123,470 / 53,028 / 315,050)
2026-09-09 17:15 UTC	d0239b5b	V165 0704eda	105,544	53,374	349,160	5.7576	제출 시점 1위 후보(직전 1위 #905 5.667)
2026-09-10 05:12 UTC	46bc9eed	V204_submit 3c381d2	101,132	56,845	317,445	5.9032	메모리 대역폭 캠페인 (V196~V204) 결과

팀명 **Goat Chovy #1557**. 채점 수치는 Arena cold 실측과 같은 급이다.

2. 실측 계보 (점수가 되는 유일한 표)

정확도 PASS를 받은 구성만 올린다. 각 행이 그 시점의 SOTA다.

브랜치	qkv	attn_out	ffn	기하평균	날짜
V0_baseline	244,885	405,253	3,706,465	1.000×	09-09
V7_qkv_x_replicate_via_hbm	190,342	111,005	1,213,877	2.430×	09-09
V14_two_clusters	183,005	78,708	418,837	3.936×	09-09
V50_drop_false_replication	149,244	56,522	354,825	4.972×	09-09
V82_ffn_up_tiles_12x5	136,576	56,586	351,454	5.135×	09-09
V158_qkv_x_ring32_broadcast	107,552	58,474	353,628	≈5.6× (V0 실측 기준; 공식 baseline 기준 ≈5.7)	09-09 (job 15566, 리더보드 제출 대기)
V165_qkv_broadcast_attnout_two_tiles	105,544	53,374	349,160	5.7576 (공식 d0239b5b)	09-09
V204_submit	101,132	56,845	317,445	5.9032 (공식 46bc9eed)	09-10

V82 아래 두 행은 **공식 채점값**(분모 = 공식 baseline 250,514 / 404,633 / 3,703,473), 그 위는 우리 실측(분모 = V0_baseline)이다. 2% 안쪽 차이로 추세를 읽는 데는 지장이 없지만 행끼리 직접 빼면 안 된다.

커널별 speedup으로 다시 보면 서사가 한눈에 들어온다.

시점	qkv	attn_out	ffn
V7	1.29×	3.65×	3.05×
V14	1.34×	5.15×	8.85×
V50	1.64×	7.17×	10.45×
V82 (공식)	1.61×	7.71×	10.59×
V165 (공식)	2.37×	7.58×	10.61×
V204 (공식)	2.48×	7.12×	11.67×

오랫동안 qkv 하나가 제자리였다 — V7에서 V82까지 1.29 → 1.61로만 움직이는 동안 다른 둘은 7~10배가 됐다. 그 벽을 깬 것이 V165(§3.6)이고, 깬 방법이 이 프로젝트에서 가장 중요한 발견이다.

측정 노이즈와 그 해결. 초기에는 같은 바이너리·같은 픽스처로도 qkv가 6~8% 흔들려 1% 개선을 판정할 수 없었다. 나중에 정체가 밝혀졌다 — **첫 실행의 명령 체크 스테이징**이다. 같은 프로세스에서 두 번째 실행부터는 qkv도 ±0.7%다. 그래서 **A/B는 한 잡 안 warm median으로, 보고는 cold로** 한다(RULES §5.4). 잡간 절대값은 비교하지 않는다 — 같은 코드가 잡에 따라 attn_out 51.7k~60.3k, ffn 311.9k~324.2k로 흔들린다.

3. 갱신 이력

시간순, 오래된 것이 위.

3.1 V0_baseline — 기준선 (1.000×

- **날짜:** 2026-09-09
- **한 줄:** 원본 skeleton에 실험 인프라(RULES/RESULTS/SOTA/requirements)만 붙인 출발점. 커널 코드 변경 없음.
- **실측:** qkv 244,885 / attn_out 405,253 / ffn 3,706,465, 3/3 PASS.
- **여기서 읽은 것:** 세 커널 모두 배치 1 행렬-벡터라 산술 강도가 2~3.5 FLOP/byte에 불과한데, 실측은 HBM 이동 하한보다도 한참 위였다. 병목이 대역폭도 연산량도 아닌 **온칩 부수 작업**이라는 뜻이고, 이후 50번의 실험이 전부 그 부수 작업을 걷어내는 일이었다.

3.2 V7_qkv_x_replicate_via_hbm — 2.430× (누적 V1+V2+V7)

- **날짜:** 2026-09-09
- **한 줄:** 세 가지를 쌓았다 — FFN down 투영을 체크 단위로만 역양자화(V1), O 투영을 32 → 256 슬라이스로 확장(V2), 그리고 x 복제를 온칩 switch가 아니라 **HBM 왕복**으로(V7).
- **병목이었던 것:** `MainContext` 96% 점유. 최장 노드가 투영이 아니라 `layout.rs`의 브로드캐스트 헬퍼(약 62k cycle)였다.
- **왜 통했는가:** 셋 다 계산식을 안 바꾸고 **데이터를 어디에 어떻게 놓을지**만 바꿨다. 특히 V7이 반직관적이다 — 온칩 브로드캐스트(54~62k)보다 HBM에 7.5 KB 썼다가 복제 로드하는 쪽이 빨랐다. "**온칩이 무조건 빠르**

다"가 틀렸다.

- 커널별 기여: qkv 1.29× / attn_out 3.65× / ffn 3.05×
- 포기한 것: 없음. 수치 연산이 바뀌지 않아 정확도 여유를 쓰지 않았다.

3.3 V14_two_clusters — 3.936× (이전 2.430× 대비 +62%)

- 날짜: 2026-09-09
- 한 줄: 칩의 두 클러스터 중 하나만 쓰고 있었다. 행을 실제로 나눠 512 슬라이스를 전부 동원했다.
- 병목이었던 것: 병목이랄 것도 없었다 — `Cluster = m![1 # 2]` 가 "논리 1개를 물리 2개 자리에 복제"라는 뜻이었고, 두 클러스터가 같은 계산을 반복하고 있었다. 타입 표기 한 줄에 숨어 있었다.
- 왜 통했는가: DMA 대역폭과 연산기가 둘 다 2배가 됐다. 세 커널이 동시에 빨라진 유일한 변경이고, 기하평균 점수 체계에서 가장 효율이 좋은 종류다.
- 커널별 기여: qkv 1.34× / attn_out 5.15× / ffn 8.85×
- 포기한 것: 클러스터 간 합류 경로가 새로 필요해졌다(HBM 경유). 이 관용구가 나중에 V15의 오해로 이어진다.
- 배운 것: V1~V13이 열세 번의 실험으로 벌어들인 것보다, 놓고 있는 자원을 켜 한 번이 더 컸다. 미세 튜닝 전에 "안 쓰고 있는 게 없나"를 먼저 본다.

3.4 V50_drop_false_replication — 4.972× (이전 3.936× 대비 +26%)

- 날짜: 2026-09-09
- 한 줄: 잘못된 전제에 기댄 것을 전부 되돌렸다. 빼서 이긴 유일한 갱신이다.
- 무엇이 틀렸었나: V15가 도입한 HBM 사본 관용구는 "소스에 없는 축을 목적지에만 붙이면 DMA write가 복제된다"를 가정했다. 복제되지 않는다. 사본 1개만 채워지고 나머지는 초기화되지 않은 HBM이라, bf16 경로는 NaN, f8 경로는 220%대 오차가 났다. V15~V41 전 계보가 정확도 FAIL이었고, makespan 5.801×의 v41은 3/3 FAIL로 기각됐다.
- 왜 되돌리는 게 답이었나: 사본을 진짜로 채우는 길(V49)을 먼저 시도했지만 막혔다. store 하나마다 Core 디스크립터 명령이 3개 붙고 Core는 in-order 발행이라, 사본 수 1/2/4/8/16 어디에서도 사본 없는 쪽을 못 이긴다(16부는 반복 하나가 5,942 cycle 간격으로 직렬화). 아이디어 자체가 틀렸으므로 남은 결론은 "없앤다"였다.
- 두 번째 버그를 찾은 방법 (재사용할 가치가 있다): 1번만 고치고 제출하니 v·attn_out·ffn은 PASS인데 q·k만 FAIL이고, non-finite 위치가 index 0에서 q=641, k=385로 옮겨갔다. Ds=256 이므로 $641 = 2 \times 256 + 129$, $385 = 1 \times 256 + 129$ — 둘 다 같은 채널 d=129, 같은 kv head n=1. v만 통과한다는 사실과 합치면 남은 경로는 RoPE뿐이고, RoPE는 Ds를 128씩 반으로 나누므로 d=129는 두 번째 half의 두 번째 원소다. 여기서 곧장 V30이 나왔다 — V30도 같은 전제로 cos/sin 테이블을 head 레이아웃에 직접 gather하고 있었다. 실패 위치의 인덱스는 커널 디버깅에서 가장 값싼 단서다.
- 커널별 기여: qkv 1.64× / attn_out 7.17× / ffn 10.45×
- 포기한 것: qkv makespan이 V38의 45,744에서 58,962로 나빠졌다(사본 상실 + V30 복원 +2,419). 그런데 실측에서는 사본이 있는 쪽이 qkv를 더 줄였다(V38 109,309 vs V50 149,244) — 무효한 숫자지만, 같은 주소를 512개 디스크립터가 읽는 부하를 정적 모델이 과소평가한다는 신호다. 올바른 사본의 실측 A/B는 아직 열려 있다.
- 이 날의 교훈: 두 버그가 같은 오해 하나에서 나왔다. 정적 스케줄에서 "공짜로 늘어난 것"은 대개 늘어나지 않은 것이다.

3.5 V158_qkv_x_ring32_broadcast — qkv 151k → 108k (-28%), 기하평균 $\approx 5.6\times$ (이전 $5.03\times$ 대비 +11%)

- **날짜:** 2026-09-09 (job 15563/15564/15566)
- **한 줄:** qkv의 x 복제 로드를 **HBM 16 디스크립터 + ring-32 switch broadcast**로 바꿨다. 데이터 이동만 바꿨고 계산식은 그대로다.
- **병목이었던 것:** 정적 스케줄이 18.4k로 세던 x 복제 로드(512 슬라이스가 같은 7.7 KB를 읽는 512 디스크립터)가 실물에서는 $\sim 45k$ 였다. 실측/makespan 비가 qkv만 2.4~2.7(다른 둘 2.05~2.2)이었던 이유이자, V138(바이트 절반)이 무효과였던 이유다.
- **왜 통했는가:** 런타임 소스(npu 백엔드)를 읽어 보니 커널은 PE의 ARM 코어 프로그램이고 DMA 디스크립터를 런타임에 직렬로 만든다. 즉 실물 비용은 디스크립터 수와 같은 영역 반복 읽기에 묶인다. 클러스터당 8부만 HBM에서 읽고(16 디스크립터) 링 32의 switch로 채우면 디스크립터 496개가 사라진다. 첫 판(V154/V155)은 cluster 1이 garbage였다 — **더미 클러스터 축 위의 pass는 cluster 0만 돈다**. 실제 축($q_s / 2048$) 위에서 로드-switch하고 끝에 Replicated로 reshape하니 PASS.
- **커널별 기여:** qkv $2.28\times$ (V0 실측 244,885 기준) / attn_out:ffn 불변.
- **포기한 것:** 없음. switch pass 7.9k(정적)는 Main이 노는 구간에 들어간다.
- **배운 것:** 이 칩에서는 "바이트"가 아니라 "DMA 디스크립터 수"를 센다. 같은 원리를 ffn/attn_out의 $x2(V159/V161)$ 와 attn_out 타일 수(V162)에 적용 중.

3.6 V165_qkv_broadcast_attnout_two_tiles — 공식 5.7576 (이전 5.0860 대비 +13%)

- **날짜:** 2026-09-09 (공식 d0239b5b)
- **한 줄:** V158의 ring-32 broadcast를 프로덕션에 올리고 attn_out O-weight를 48/12 두 타일로. **qkv 155k → 106k**.
- **왜 이게 결정적이었나:** V158($\S 3.5$)이 찾아낸 것은 최적화 하나가 아니라 **비용 모델 자체**였다. "실물 비용은 바이트가 아니라 DMA 디스크립터 수" — 이 렌즈를 얻고 나니 그동안 설명되지 않던 것들이 한꺼번에 풀렸다. 정적 18.4k가 실물 45k였던 것, 실측/makespan 비가 qkv에서만 2.4~2.7이었던 것, **바이트를 절반으로 줄여도 안 변했던 것(V138)**, 사본을 늘리면 오히려 느려졌던 것(V52)이 전부 같은 원인이었다.
- **커널별 기여:** qkv **2.37 $\times$** (V82의 1.61 $\times$ 에서) / attn_out:ffn 사실상 불변.
- **포기한 것:** 없음.
- **배운 것:** 이 시점에서 세 커널이 전부 **~ 290 B/cycle**에 붙었다(298 / 295 / 285). 서로 다른 코드가 서로 다른 최적화를 거쳐 5% 안에 모였다는 것은 **공통의 물리적 한계**를 만났다는 뜻이다. 여기서부터 "무엇을 덜 계산할까"는 끝나고 "바이트를 어떻게 덜 읽거나 더 연속적으로 읽을까"만 남았다.

3.7 V182 → V194 — ffn 세그먼트 통합 (Arena 5.86 → 6.05 기대)

- **날짜:** 2026-09-09 18:35 ~ 2026-09-10 00:20 (Arena 후보, 공식 미제출)
- **한 줄:** ffn up/gate를 슬라이스당 30행 $\times$ H 전체로 읽어 가중치-스케일을 각각 **연속 1 세그먼트**로 만들었다. **ffn 349k → 318k (-8.9%)**.
- **병목이었던 것:** 블록 스케일을 120 B 세그먼트 60개씩 읽고 있었다. 탐침(V174)이 먼저 값을 잰다 — **연속 3.1 cycle/KB vs 세그먼트 6.6 cycle/KB**, 즉 세그먼트 로드는 바이트당 2배다. 프로덕션의 스케일 로드 3개가 ffn의 21%($\approx 73k$)였다.
- **왜 통했는가:** $\S 3.6$ 의 벽 앞에서 남은 레버가 "바이트를 더 연속적으로"뿐이었고, 이게 정확히 그것이다.

- **함께 확인된 것:** ffn에서 계산을 빼봤더니(V170) post-FF tail은 **-0.2k**, geglu 체인은 +1.6k, 스케일 로드 하나는 +7k였다. **계산을 지워도 ffn은 안 줄어든다** — 99 MB를 흘리는 시간이 전부다.
- **V194가 더한 것:** RoPE cos/sin 스테이징 버퍼 공유 + cos 곱을 sin 덧셈에 융합(qkv **-3.6%, 5/5 잡 승**), O-weight 행 분할 44/16.
- **포기한 것:** ffn down에도 같은 수법을 시도했으나 **+27k로 기각**(V183/V184) — x broadcast 480패킷과 f32 hop 비용이 세그먼트 이득을 넘었다.

3.8 V204_submit — 공식 5.9032 (이전 5.7576 대비 +2.5%)

- **날짜:** 2026-09-10 05:12 UTC (공식 46bc9eed). 측정본은 V200_production.
- **한 줄: 메모리 대역폭 캠페인**(V196~V203, Arena 41슬롯)의 결과. 두 가지가 살아남았다 — attn_out의 **256 B** 런 정렬과 ffn의 스테이징 store 병합.
- **왜 통했는가 (1) — DMA 비용 곡선을 실측했다.** V197이 런 길이를 스위치 곡선을 그렸다: **256 B 런이 최적** (618 B/cycle)이고, 그보다 짧으면 **정확히 2× 벌점**, 길면 완만히 나빠진다. §3.6의 "290 B/cycle 벽"은 벽이 아니라 런 길이가 잘못된 지점의 값이었던 것이다. 다만 256 B로 옮기려면 분할 축이 2의 거듭제곱이어야 하는데 **attn_out만 가능하다**(Qs=4096; 다른 둘은 H=3840).
- **왜 통했는가 (2) — DMA 명령 병합은 위치가 전부다.** 명령 수가 아니라 그 명령이 커널 선두에서 weight 스트림을 막고 있는지로 값이 정해진다. ffn 선두에서 store 2개를 1개로 합치니 **-11,078 (-3.5%, 6/6 잡 승)**, 같은 병합을 중반에서 하면 0이다(V201이 반례).
- **커널별 기여:** qkv 2.48× / attn_out 7.12× / ffn **11.67×**
- **포기한 것:** 없음. 다만 공식 **draw가 나뉘었다** — Arena 실측은 6.229(qkv 100.3k / attn_out 49.9k / ffn 310.1k) 인데 공식 채점에서 attn_out이 56,845로 나왔다(같은 코드의 Arena 범위 47.9k~52.0k). **코드 차이가 아니라 cold 1회 draw의 분산이다.**
- **닫힌 길:** 디바이스 팬아웃(chip=1 이 이미 8 PE), DMA 병렬성(DmaEngine 1개), 바이트 축소(픽스처가 PRNG 난수 가중치라 압축 불가) — 전부 확인 후 닫았다.

4. makespan 사다리 (참고용 — 점수 아님)

첫 실측 전까지 이 표가 SOTA 역할을 했다. 지금은 기록으로만 남긴다.

구간	상태
V1 ~ V14	정확도 PASS 확인됨 (V7~V14는 실측 SOTA)
V15 ~ V41	전부 정확도 FAIL. HBM 사본 관용구에 오염됨 (§3.4)
V50	58,962 / 27,424 / 158,182 = makespan 5.310× (실측 4.972×)

▶ 전체 사다리 (V7 → V41, 27칸)

V16~V37의 최적화 대부분은 사본 관용구와 무관하고 정확도 위험도 없다. V50 위에 다시 쌓을 수 있다. 다만 채택 판정은 실측으로만 한다 — 이 표가 왜 그래야 하는지의 증거다.

5. 서사 메모 (report용)

최종 리포트에서 쓸 관점. 시간순이 아니라 무게순이다.

5.1 이 프로젝트를 한 문장으로

곱셈은 한 번도 줄이지 않았다. V0에서 V204까지 5.9배가 나왔는데 계산식은 그대로다. 바뀐 것은 데이터를 어떤 타입으로, 어디에, 몇 조각으로 놓고, **몇 번에 나눠 옮기는가**뿐이다. 배치 1 LLM 디코드는 산술 강도가 2~3.5 FLOP/byte로 RNGD의 균형점(약 170)에 한참 못 미치는 **메모리 바운드** 워크로드이고, 그런 곳에서는 배치가 전부다.

5.2 가장 비싸게 배운 것 — makespan은 점수가 아니다

정적 스케줄에서 27칸을 5.801×까지 올려놓고 첫 실측에서 **위쪽 절반이 통째로 무효화**됐다. 원인은 성능 모델이 아니라 **정확도**였다. 정적 도구는 "이 코드가 얼마나 걸리는가"에 답하지 **"이 코드가 맞는가"에 답하지 않는다.**

같은 오해가 두 곳에서 터졌다. 사본 관용구(V15)와 RoPE 테이블 gather(V30)는 다른 시점에 쓴 코드인데 전제가 같았다 — **"소스에 없는 축은 복제된다"**. 정적 스케줄에서 공짜로 늘어난 것처럼 보이는 것은 대개 늘어나지 않은 것이다.

다만 **makespan을 버리라는 뜻은 아니다.** V204의 채택 근거도 결국 정적 예측이었다. 쓸 수 있는 것과 없는 것이 갈릴 뿐이다 — 방향에는 답하고, 정확성과 (노이즈에 묻히는) 크기에는 답하지 않는다.

5.3 가장 중요한 발견 — 실물 비용은 바이트가 아니다

PE 안의 커널은 ARM 코어에서 도는 프로그램이고, DMA 디스크립터를 런타임에 직렬로 만들어 발행한다. 그러니 비용의 축이 둘이다: 옮기는 **바이트**(대역폭)와 발행하는 **디스크립터 개수**(코어의 직렬 작업).

이걸 모르는 동안 qkv 하나가 몇십 번의 실험에 반응하지 않았다. 결정적 실마리는 **"바이트를 절반으로 줄였는데 시간이 안 줄었다"**(V138)였다 — 시간을 먹는 것이 바이트가 아니라는 뜻이기 때문이다. 답은 스케줄러가 아니라 런타임 소스에 있었다.

그리고 이 렌즈가 나머지 전부를 다시 보게 했다: 세그먼트 로드가 바이트당 2배라는 것(V174), ffn이 계산이 아니라 스트림에만 묶여 있다는 것(V170), 출력 gather는 오히려 비싸다는 것(V167/V168), **256 B 런이 최적이고 그 미만은 정확히 2× 벌점**이라는 것(V197).

5.4 세 번의 큰 갱신이 전부 튜닝이 아니었다

갱신	무엇을 했나	성격
V14 (+62%)	놓고 있던 클러스터를 컸다	자원 미사용
V50 (+26%)	틀린 전제에 기댄 것을 지웠다	버그 수정
V165 (+13%)	비용 모델을 잘못 알고 있었음을 발견했다	모델 오류

미세 튜닝 수십 번보다 이 셋이 컸다. 튜닝 전에 **"안 쓰고 있는 게 없나 / 틀린 게 없나 / 내가 세는 단위가 맞나"**를 먼저 본다.

5.5 측정 자체가 연구 대상이었다

초기에는 같은 코드가 qkv에서 $\pm 8\%$ 흔들려 1% 개선을 판정할 수 없었다. 정체는 **첫 실행의 명령 체크 스테이징** 이었고, warm은 $\pm 0.7\%$ 다. **A/B는 한 잡 안 warm median, 보고는 cold** — 이 구분을 세우고 나서야 1% 단위 실험이 가능해졌다.

그리고 **측정 예산이 최적화의 해상도를 정한다**. 공식 제출은 하루 5회, Arena 잡도 유한하다. 그래서 한 잡에 변형 여러 개를 warm 반복으로 담는 하네스를 만들었고, 그게 후반 캠페인(V196~V203, 41슬롯)을 가능하게 했다.

5.6 그 밖에

점수 체계가 전략을 규정한다. 기하평균이라 한 커널의 극단적 최적화보다 균형 잡힌 개선이 유리하고, 어느 커널을 2배 만들든 효과가 같으므로 "제일 느린 곳"이 아니라 "**제일 쉽게 2배 되는 곳**"을 골라야 한다.

공유 코드는 레버리지와 위험이 같은 손잡이에 달려 있다. `device/shared/*` 는 세 커널이 모두 쓰므로 효과가 곱해지지만 A/B 해석과 Stage 2 회귀 위험도 함께 커진다.

정밀도 여유는 커널마다 다르고, 쓰인 방식이 예상과 달랐다. `decoder_feedforward (0.01)`가 가장 뽀뽀하고 `sliding_attention_output (0.05)`이 가장 험겁다. 다만 이 여유는 정밀도를 낮추는 데가 아니라 **수학적으로 동일한 재배치를 허가받는** 데 쓰였다(V29의 부분합 스케일링, V33의 상수 스케일).

6. 지금 서 있는 자리

공식 5.9032, Arena 6.229. 공식 draw에서 `attn_out`이 나쁘게 나온 것(56,845 vs Arena 47.9k~52.0k)이라 **같은 코드로 재제출만 해도 6.0대가 기대**되지만, 제출 예산은 하루 5회다(RULES §5.4).

세 커널의 현재 실효 전송률 (V200 Arena 기준):

커널	바이트	cycle	B/cycle
qkv	30.0 MB	100,314	314
attn_out	15.0 MB	49,928	315
ffn	94.9 MB	310,088	321

V197이 켜 최적점은 **618 B/cycle(256 B 런)**이다. 즉 **아직 2배가 남아 있다** — 다만 256 B 정렬은 분할 축이 2의 거듭제곱일 때만 되고, 그게 되는 것은 `attn_out(Qs=4096)`뿐이다. **qkv·ffn은 H=3840이라 막혀 있다**.

열려 있는 것. - 재제출로 좋은 **draw 확보** — 코드 변경 없이 기대 +2%. - **H=3840을 2의 거듭제곱 조각으로 재배치할 수 있는가** — 되면 qkv·ffn에도 256 B 런이 열린다. 이게 남은 가장 큰 축이다. - **DMA 명령 병합을 다른 커널 선두에도** — 값은 위치가 정한다(V201).

닫힌 것 (다시 시도하지 말 것). 디바이스 팬아웃(`chip=1` 이 이미 8 PE), DMA 병렬성(DmaEngine 1개), 바이트 축소(픽스처가 PRNG 난수라 압축 불가), ffn down 세그먼트 통합(V183/V184, +17.7k), 출력 gather 후 store(V167/V168), HBM 사본(V49/V52).